Laboratório de Concorrência · Aula 10

Paulo Henrique Malavasi · RA 24368074-2 · Turma C
Gerado em 10/10/2026, 20:52:43 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização por cooperação acontece quando duas pessoas dependem uma da outra para realizar uma tarefa, como um garçom esperando o cozinheiro preparar o pedido.
Já a sincronização por competição acontece quando duas pessoas querem usar o mesmo recurso, como duas pessoas esperando para usar o mesmo banheiro ou bebedor.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

1.000.000

1ª execução (n = 1.000.000)

1019975

2ª execução (n = 1.000.000)

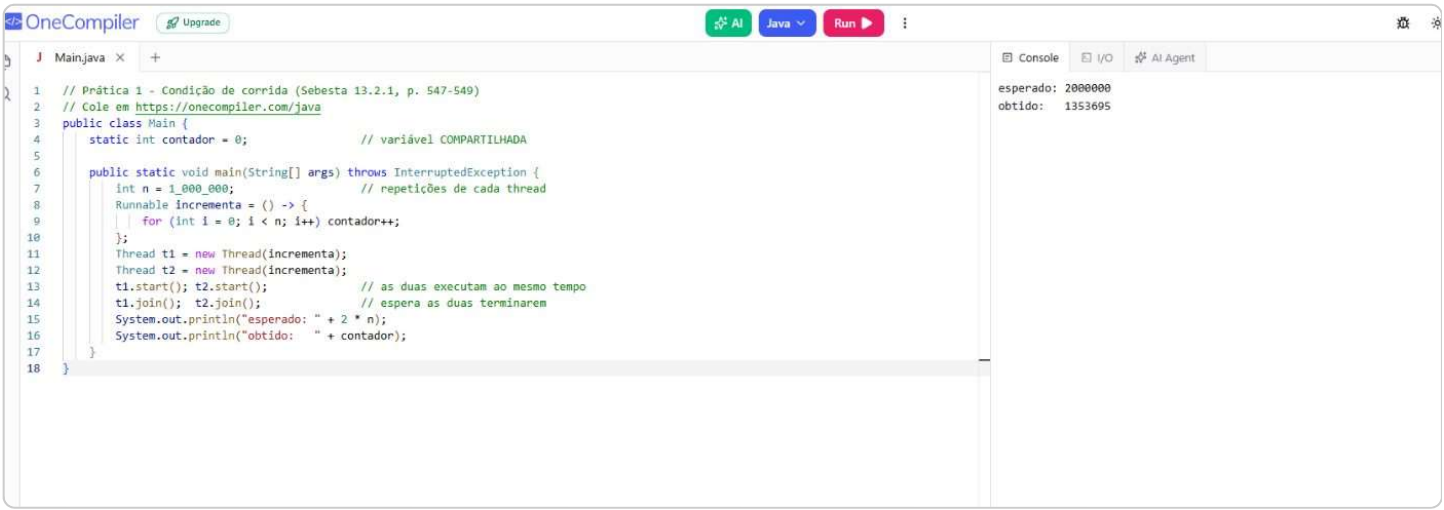
1022175

3ª execução (n = 1.000.000)

1353695

Saída do programa

não respondido




```java
// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
// Cole em https://onecompiler.com/java
public class Main {
    static int contador = 0;                // variável COMPARTILHADA

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;                  // repetições de cada thread
        Runnable incrementa = () -> {
            for (int i = 0; i < n; i++) contador++;
        };
        Thread t1 = new Thread(incrementa);
        Thread t2 = new Thread(incrementa);
        t1.start(); t2.start();             // as duas executam ao mesmo tempo
        t1.join();  t2.join();              // espera as duas terminarem
        System.out.println("esperado: " + 2 * n);
        System.out.println("obtido:   " + contador);
    }
}
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, os valores não forma iguais, o valor pode mudar de execução para execução pois são executadas ao mesmo tempo.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Thread 1 Thread 2
Lê 10 Lê 10
Soma 1 = 11 Soma 1 = 11
Grava 11 Grava 11
Sendo que o correto seria 12

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

O valor obtido foi 2000, pois como o valor é muito menor a chance de ter conflito fica menor, a condição de corrida fica mais perigoso em sistemas reais por conta do volume.

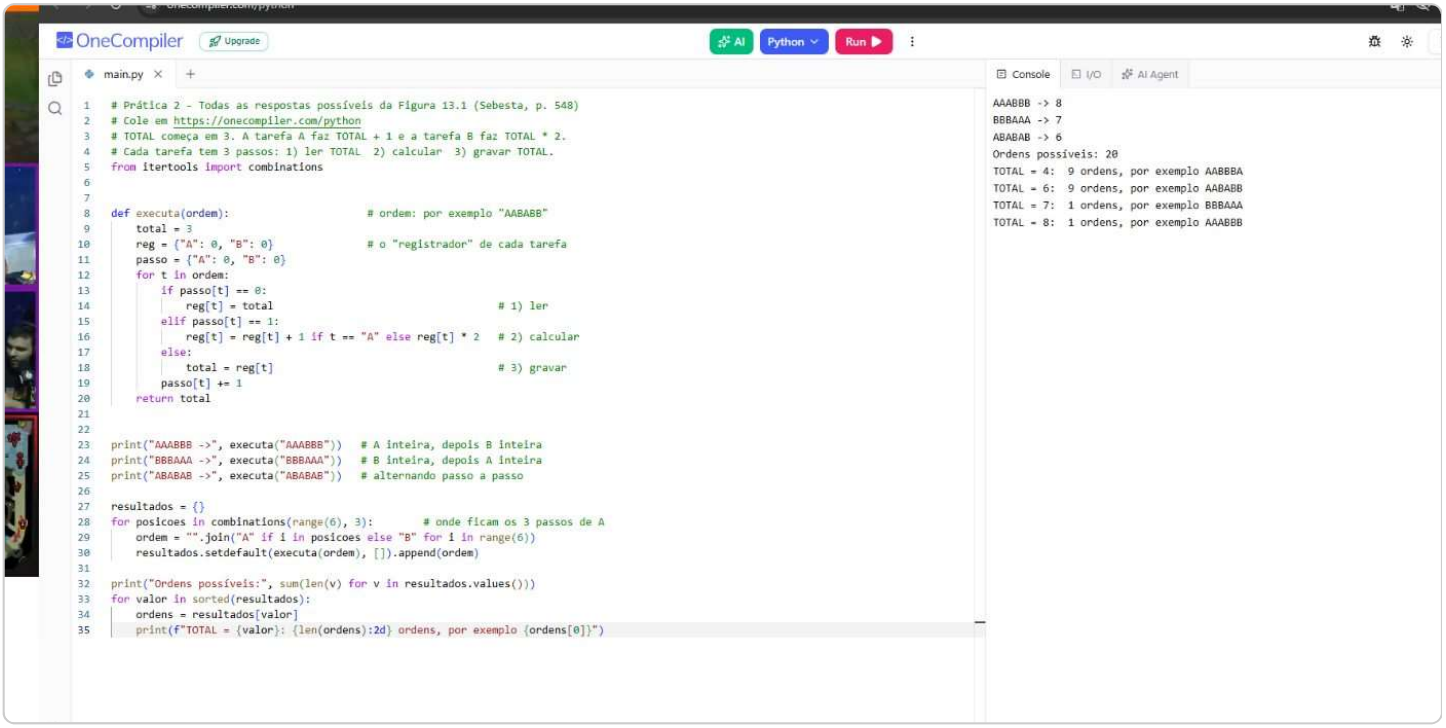
Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

não respondido

Saída do programa

não respondido



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis, que geram 4 resultados diferentes: 4, 6, 7 e 8.
Apenas 1 ordem (AAABBB) gera o resultado correto, que é 8.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

A ordem AABBBA
1 A: lê o TOTAL = 3
2 A: calcula 3 + 1 = 4, mas ainda não grava

3 B: lê o TOTAL = 3

4. B: calcula 3 × 2 = 6

5. B: grava 6 no total

6 A: grava 4 no total

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Operação atômica, Monitor será a incorreta.

Saída do programa

não respondido




```java
// Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
// Cole em https://onecompiler.com/java
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static int semCuidado = 0, comSync = 0;
    static AtomicInteger atomico = new AtomicInteger();

    static synchronized void incSync() { comSync++; }   // uma thread por vez

    static long mede(Runnable corpo) throws InterruptedException {
        Thread a = new Thread(corpo), b = new Thread(corpo);
        long inicio = System.nanoTime();
        a.start(); b.start(); a.join(); b.join();
        return (System.nanoTime() - inicio) / 1_000_000;  // milissegundos
    }

    public static void main(String[] args) throws InterruptedException {
        int n = 1_000_000;
        long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
        long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
        long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
        System.out.println("esperado:              " + 2 * n);
        System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
        System.out.println("synchronized:      " + comSync + " em " + ms2 + " ms");
        System.out.println("AtomicInteger:     " + atomico.get() + " em " + ms3 + " ms");
    }
}
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

synchronized, AtomicInteger, as versões corretas costumam ser mais lentas porque precisam controlar o acesso das threads à variável, evitando perda de dados.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, porque o AtomicInteger garante apenas que cada saldo seja alterado de forma segura, mas não garante que a transferência inteira aconteça corretamente. Pode ocorrer um erro após retirar o dinheiro de uma conta e antes de depositar na outra.

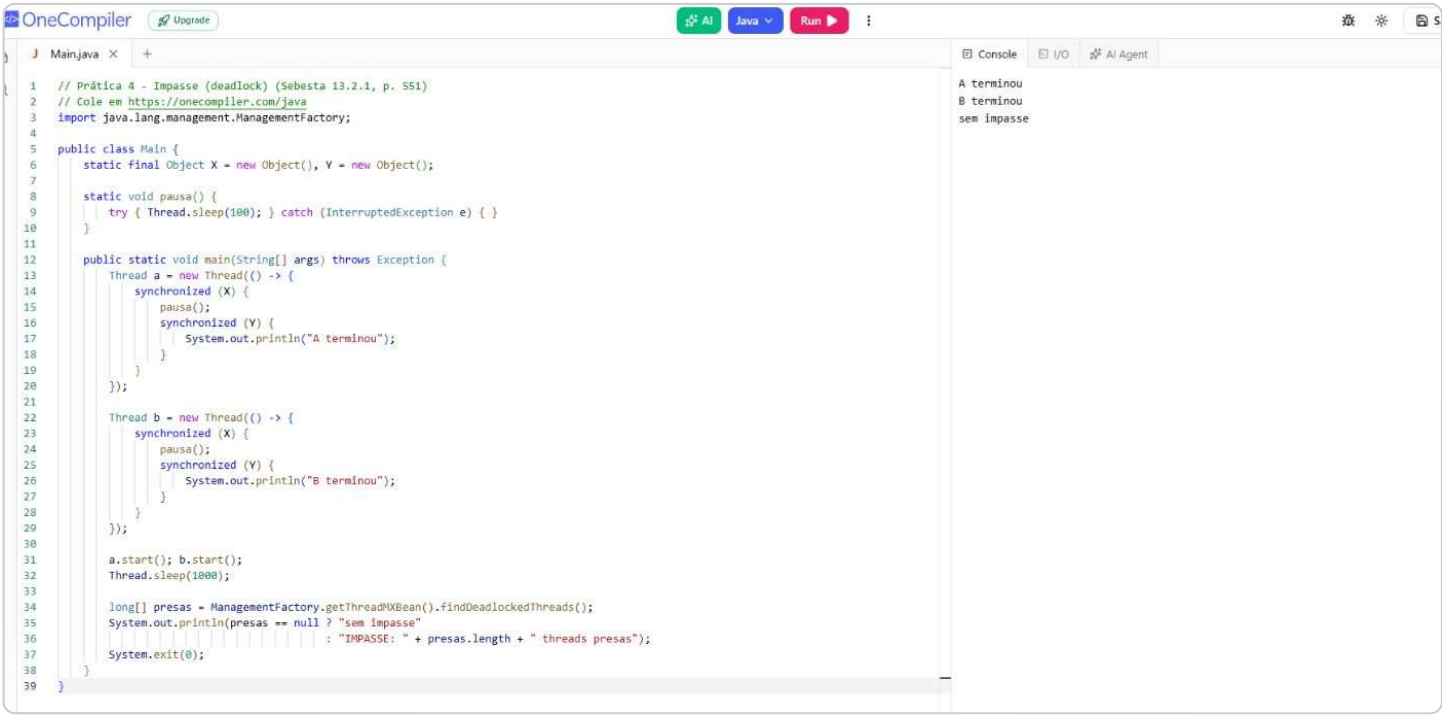
Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

A principio não, pois existe um impasse

Saídas (antes e depois da correção)

não respondido




```java
// Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
// Cole em https://onecompiler.com/java
import java.lang.management.ManagementFactory;

public class Main {
    static final Object X = new Object(), Y = new Object();

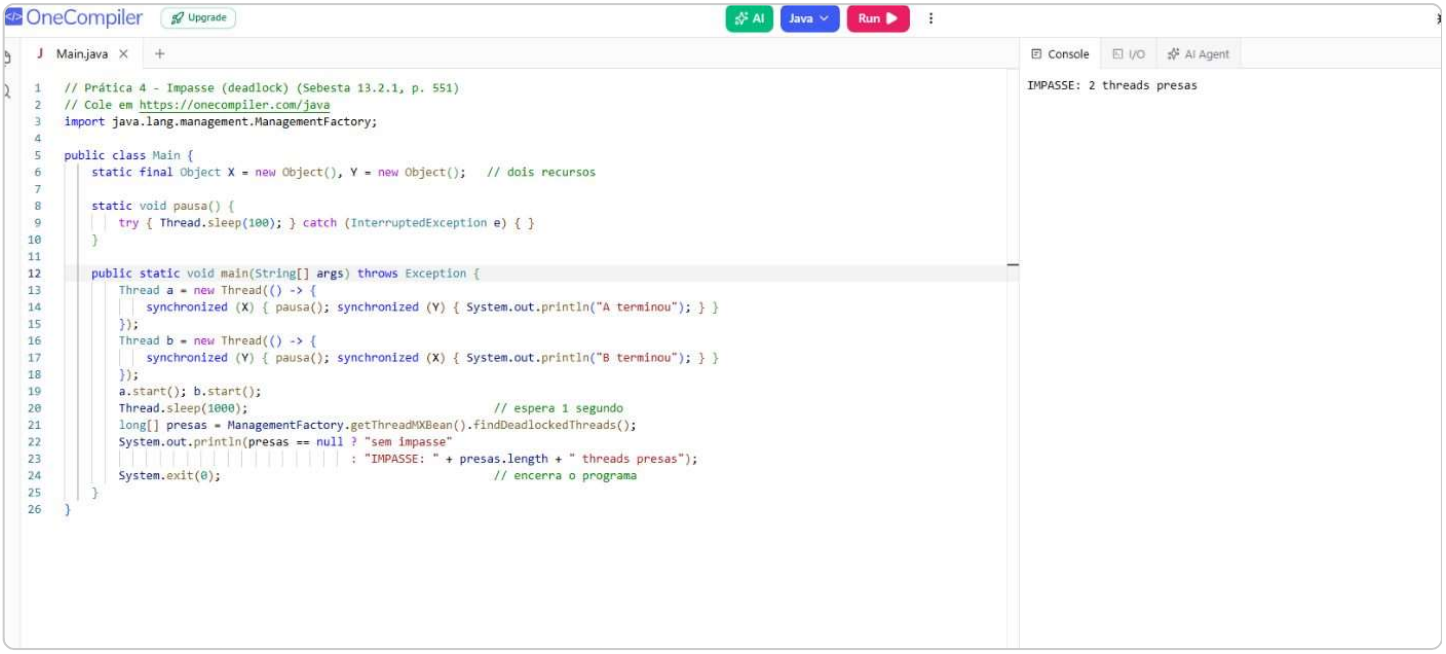
    static void pausa() {
        try { Thread.sleep(100); } catch (InterruptedException e) { }
    }

    public static void main(String[] args) throws Exception {
        Thread a = new Thread(() -> {
            synchronized (X) {
                pausa();
                synchronized (Y) {
                    System.out.println("A terminou");
                }
            }
        });

        Thread b = new Thread(() -> {
            synchronized (X) {
                pausa();
                synchronized (Y) {
                    System.out.println("B terminou");
                }
            }
        });

        a.start(); b.start();
        Thread.sleep(1000);

        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println(presas == null ? "sem impasse"
                                          : "IMPASSE: " + presas.length + " threads presas");
        System.exit(0);
    }
}
```




```java
// Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
// Cole em https://onecompiler.com/java
import java.lang.management.ManagementFactory;

public class Main {
    static final Object X = new Object(), Y = new Object();   // dois recursos

    static void pausa() {
        try { Thread.sleep(100); } catch (InterruptedException e) { }
    }

    public static void main(String[] args) throws Exception {
        Thread a = new Thread(() -> {
            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
        });
        Thread b = new Thread(() -> {
            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
        });
        a.start(); b.start();
        Thread.sleep(1000);                                    // espera 1 segundo
        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
        System.out.println(presas == null ? "sem impasse"
                                          : "IMPASSE: " + presas.length + " threads presas");
        System.exit(0);                                        // encerra o programa
    }
}
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque a thread A tinha o recurso X e esperava pelo Y, enquanto a thread B tinha o recurso Y e esperava pelo X.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita que as threads fiquem esperando uma pela outra, eliminando a espera circular e, assim, o deadlock.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

total 200 ms

1 - 600 ms

6 - 100 ms

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

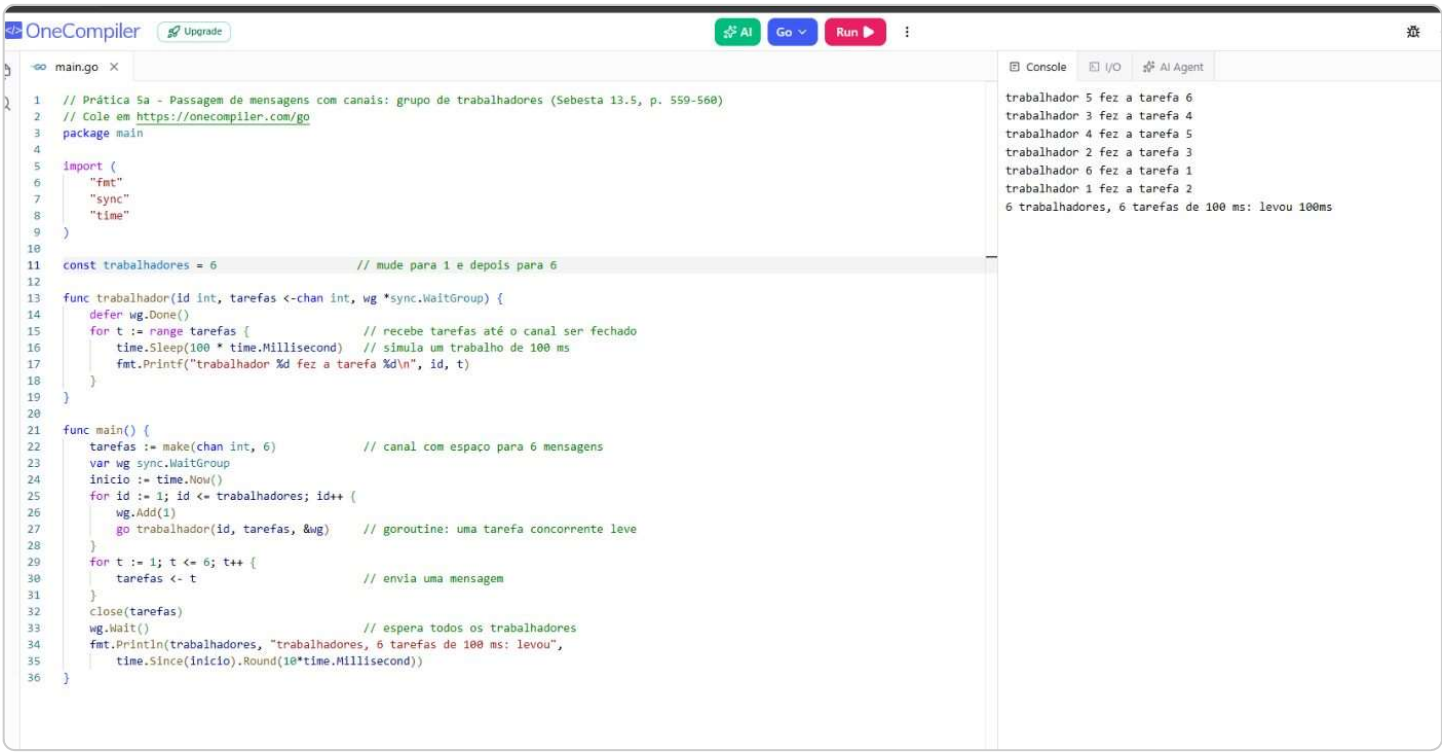
200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido



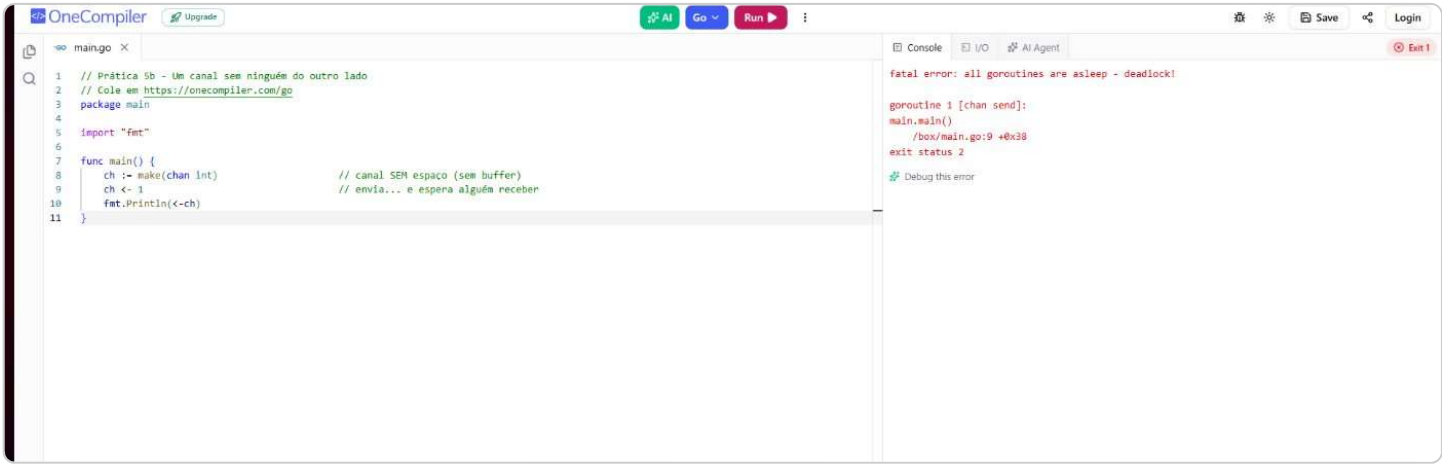
5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o tempo é de 600 ms, pois ele executa todas as tarefas sozinho. Com 3 trabalhadores, cai para 200 ms, porque cada um realiza 2 tarefas. Com 6 trabalhadores, leva 100 ms, pois todas são executadas ao mesmo tempo.

O programa não precisa de synchronized nem de travas porque os trabalhadores se comunicam por canais, sem precisar modificar uma variável compartilhada ao mesmo tempo.

Saídas (antes e depois da troca)

não respondido





5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar mensagens e precisa que alguém receba o valor para continuar. Como isso não acontece, ele fica bloqueado.

Já o canal com espaço para 1 mensagem permite armazenar o valor sem precisar de um receptor imediato. Assim, o programa consegue continuar e mostrar o resultado 1.

Prática 6: O compilador como guarda

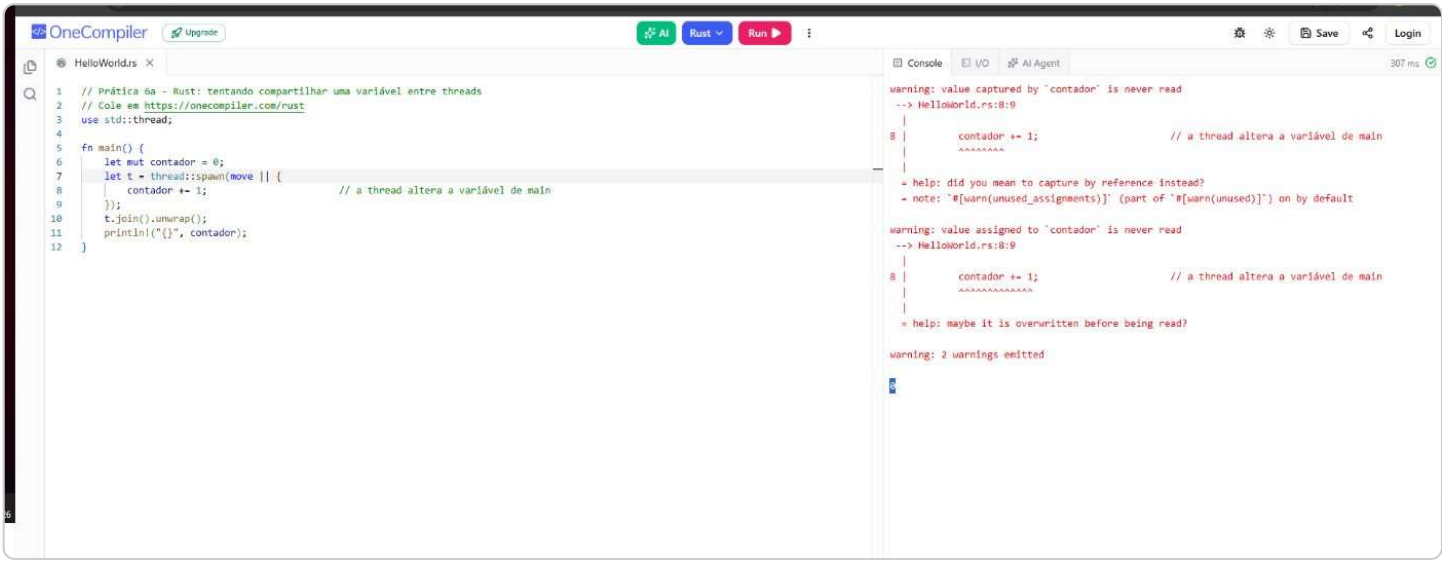
Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Provavelmente não vai compilar.

Saídas (o erro e o resultado com move)

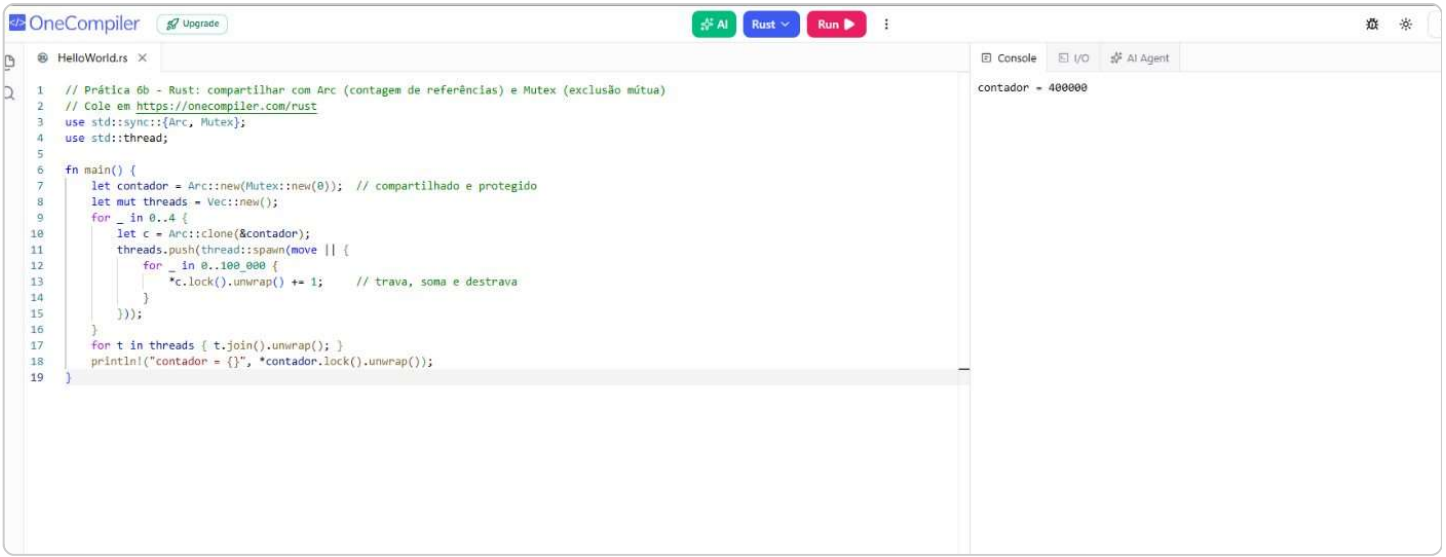
não respondido





Saída do programa

não respondido



6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

O move faz a thread trabalhar com uma cópia do valor, sem alterar a variável original. Por isso, o contador da main continua em 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro foi percebido durante a execução, quando o contador apresentou um resultado incorreto. Já em Rust, o compilador identifica o acesso inseguro antes de executar o programa.
A vantagem do Rust é evitar muitos erros de concorrência. O custo é que o programador precisa seguir regras mais rígidas e escrever código com mecanismos de proteção, como Arc e Mutex.

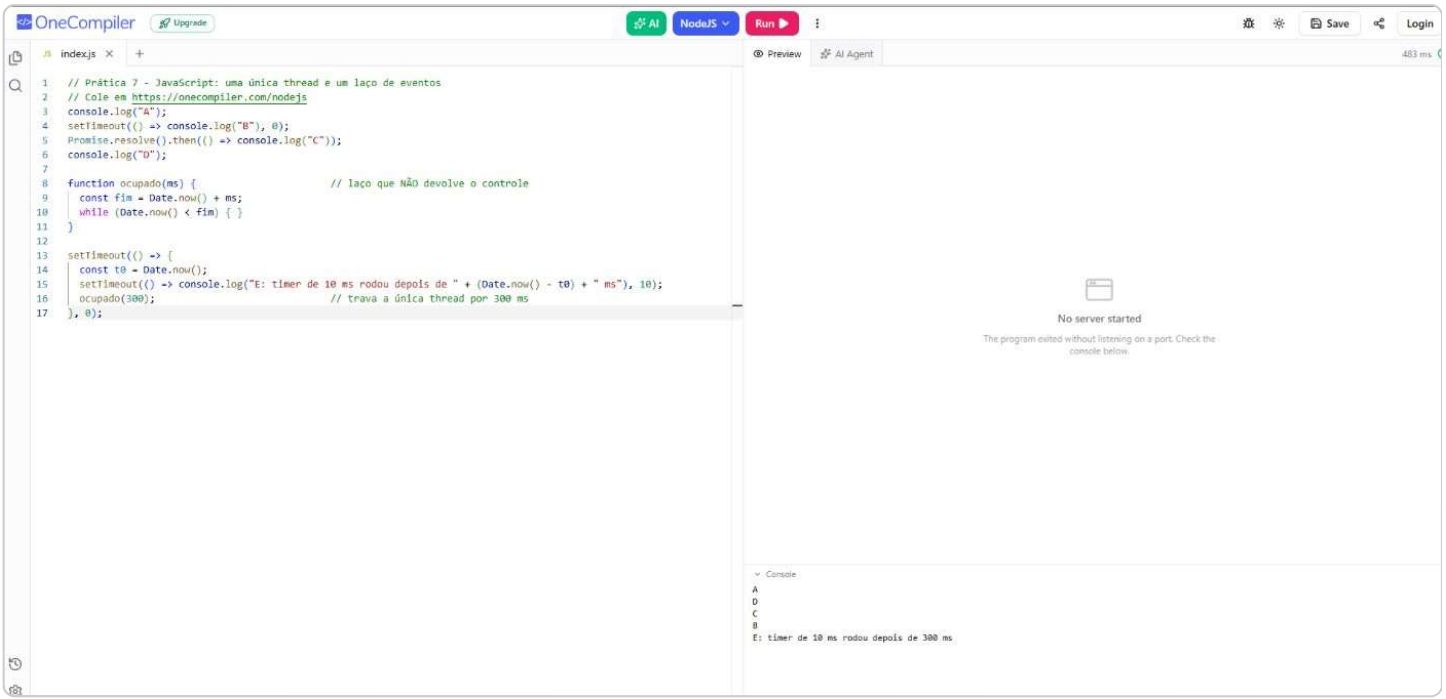
Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A - D - B - C

Saída do programa

não respondido



7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A
D
C
B
E

Isso acontece porque o JavaScript executa primeiro o código síncrono (A e D), depois as microtarefas das Promises (C) e, por último, os timers (B e E)

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer atrasou porque o cálculo de 300 ms bloqueou a thread principal, impedindo a execução de outras tarefas. Em um servidor Node.js, isso significa que um cálculo pesado pode atrasar as respostas para vários usuários, deixando o sistema lento.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode acontecer uma condição de corrida, em que as duas pessoas compram o mesmo ingresso ao mesmo tempo. Para evitar isso, usaria uma transação no banco de dados com controle de concorrência, garantindo que apenas uma pessoa consiga comprar o último ingresso.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A questão do valor obtido no contador com 2 threads somando na mesma variável.